

# Software Engineering

# Agenda

- ✓ Introduction To Software Engineering
  - Software Processes
  - Rapid Software Development
  - Practice You Should Know
  - Case Study

# **An Introduction to Software Engineering**

# FAQs about software engineering

- What is software?
- What is software engineering?
- What is the difference between software engineering and computer science?
- What is the difference between software engineering and system engineering?
- What is a software process?
- What is a software process model?

# FAQs about software engineering

- What are the costs of software engineering?
- What are software engineering methods?
- What is CASE (Computer-Aided Software Engineering)
- What are the attributes of good software?
- What are the key challenges facing software engineering?

# What is Software?

- Computer programs and associated documentation such as
- requirements, design models and user manuals.
- Software products may be developed for a particular customer or may be developed for a general market.
- **Software products may be:**
  - Generic - developed to be sold to a range of different customers e.g. PC software such as Excel or Word.
  - Bespoke (custom) - developed for a single customer according to their specification.
- New software can be created by developing new programs, configuring generic software systems or reusing existing software.

# What is Software engineering?

- Software engineering is an engineering discipline that is concerned with all aspects of software production.
- Software engineers should adopt a systematic and organized approach to their work and use appropriate tools and techniques depending on the problem to be solved, the development constraints and the resources available.

# What is the difference between software engineering and computer science?

- Computer science is concerned with theory and fundamentals; software engineering is concerned with the practicalities of developing and delivering useful software.
- Computer science theories are still insufficient to act as a complete underpinning for software engineering (unlike e.g. physics and electrical engineering).



## What is the difference between software engineering and system engineering?

- System engineering is concerned with all aspects of computer-based systems development including hardware, software and process engineering. Software engineering is part of this process concerned with developing the software infrastructure, control, applications and databases in the system.
- System engineers are involved in system specification, architectural design, integration and deployment.

# What is a software process?

- A set of activities whose goal is the development or evolution of software.
- Generic activities in all software processes are:
  - **Specification:-** what the system should do and its development constraints
  - **Development:-** production of the software system
  - **Validation:-** checking that the software is what the customer wants
  - **Evolution:-** changing the software in response to changing demands.

# What is a software process model?

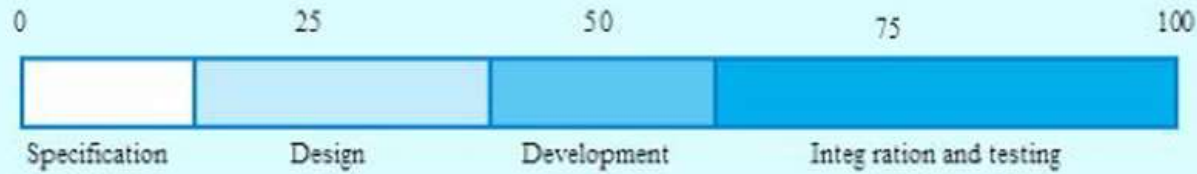
- A simplified representation of a software process, presented from a specific perspective.
- Examples of process perspectives are:-
  - Workflow perspective - sequence of activities
  - Data-flow perspective - information flow
  - Role/action perspective - who does what
- Generic process models:-
  - Waterfall
  - Iterative development
  - Component-based software engineering

# What are the costs of software engineering?

- Roughly 60% of costs are development costs, 40% are testing costs. For custom software, evolution costs often exceed development costs.
- Costs vary depending on the type of system being developed and the requirements of system attributes such as performance and system reliability.
- Distribution of costs depends on the development model that is used.

# Activity cost distribution

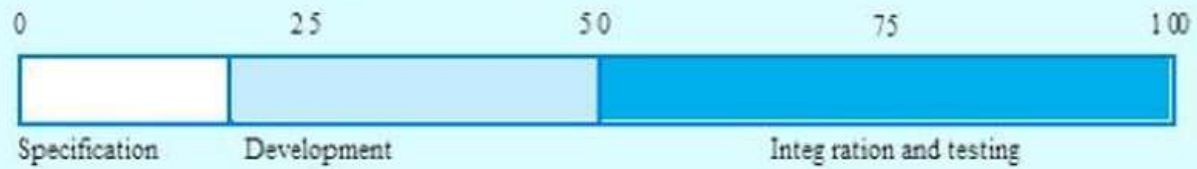
Water fall model



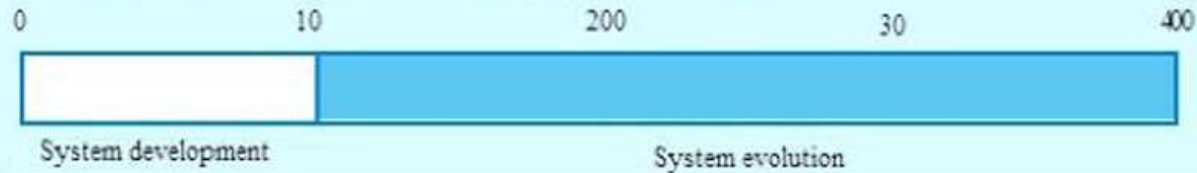
Iterative development



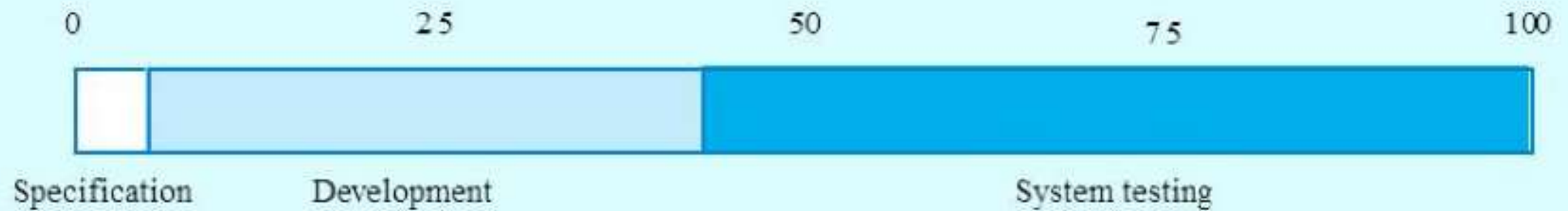
Component-based software engineering



Development and evolution costs for long-lifetime systems



# Products development costs



# What are software engineering methods?

- Structured approaches to software development which include system models, notations, rules, design advice and process guidance.
- Model descriptions
  - Descriptions of graphical models which should be produced
- Rules
  - Constraints applied to system models
- Recommendations
  - Advice on good design practice
- Process guidance
  - What activities to follow

# What is CASE (Computer-Aided Software Engineering)

- Software systems that are intended to provide automated support for software process activities.
- **Upper-CASE**
  - Tools to support the early process activities of requirements and design.
- **Lower-CASE**
  - Tools to support later activities such as programming, debugging and testing.



# What are the attributes of good software?

- The software should deliver the required functionality and performance to the user and should be maintainable, dependable and acceptable.
- Maintainability
  - Software must evolve to meet changing needs.
- Dependability
  - Software must be trustworthy.
- Efficiency
  - Software should not make wasteful use of system resources;
- Acceptability
  - Software must be accepted by the users for which it was designed. This means it must be understandable, usable and compatible with other systems.

# What are the key challenges facing software engineering?

Heterogeneity, delivery and trust

- **Heterogeneity**

- Developing techniques for building software that can cope with heterogeneous platforms and execution environments.

- **Delivery**

- Developing techniques that lead to faster delivery of software

- **Trust**

- Developing techniques that demonstrate that software can be trusted by its users.

# Professional and ethical responsibility

- Software engineering involves wider responsibilities than simply the application of technical skills.
- Software engineers must behave in an honest and ethically responsible way if they are to be respected as professionals.
- Ethical behavior is more than simply upholding the law.

# Issues of professional responsibility

- **Confidentiality**

- Engineers should normally respect the confidentiality of their employers or clients irrespective of whether or not a formal confidentiality agreement has been signed.

- **Competence**

- Engineers should not misrepresent their level of competence. They should not knowingly accept work which is outwith their competence.

# Issues of professional responsibility

- **Intellectual property rights**

- Engineers should be aware of local laws governing the use of intellectual property such as patents, copyright, etc. They should be careful to ensure that the intellectual property of employers and clients is protected.

- **Computer misuse**

- Software engineers should not use their technical skills to misuse other people's computers. Computer misuse ranges from relatively trivial (game playing on an employer's machine, say) to extremely serious (dissemination of viruses).

# Agenda

- Introduction to Software Engineering
- ✓ Software Processes
- Rapid software development
- Practices you should Know
- Case Study

# Software Processes

# Software Processes

A structured set of activities required to develop a software system

**Specification**

**Design**

**Validation**

**Evolution**

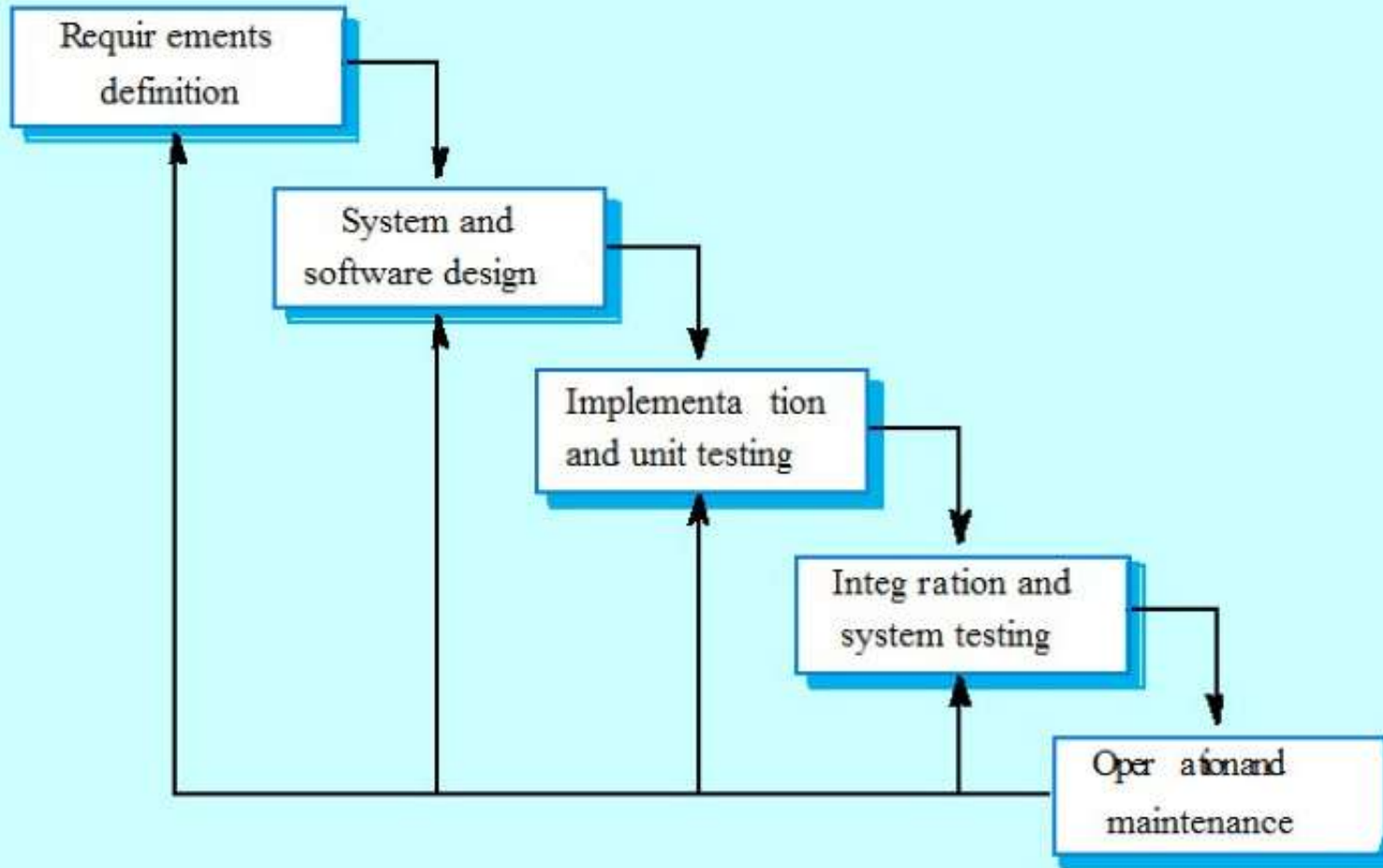
- A software process model is an abstract representation of a process.
- It presents a description of a process from some particular perspective as you will see later.



# Generic software process models

- The waterfall model
  - Separate and distinct phases of specification and development.
- Evolutionary development
  - Specification, development and validation are interleaved.
- Component-based software engineering
  - The system is assembled from existing components.

# Waterfall Model



# Waterfall model phase

1. Requirements analysis and definition
  2. System and software design
  3. Implementation and unit testing
  4. Integration and system testing
  5. Operation and maintenance
- The main drawback of the waterfall model is the difficulty of accommodating change after the process is underway. One phase has to be complete before moving onto the next phase.

# Waterfall model problem

- Inflexible partitioning of the project into distinct stages makes it difficult to respond to changing customer requirements.
- Therefore, this model is only appropriate when the requirements are well-understood and changes will be fairly limited during the design process.
- Few business systems have stable requirements.
  - The waterfall model is mostly used for large systems engineering projects where a system is developed at several sites.

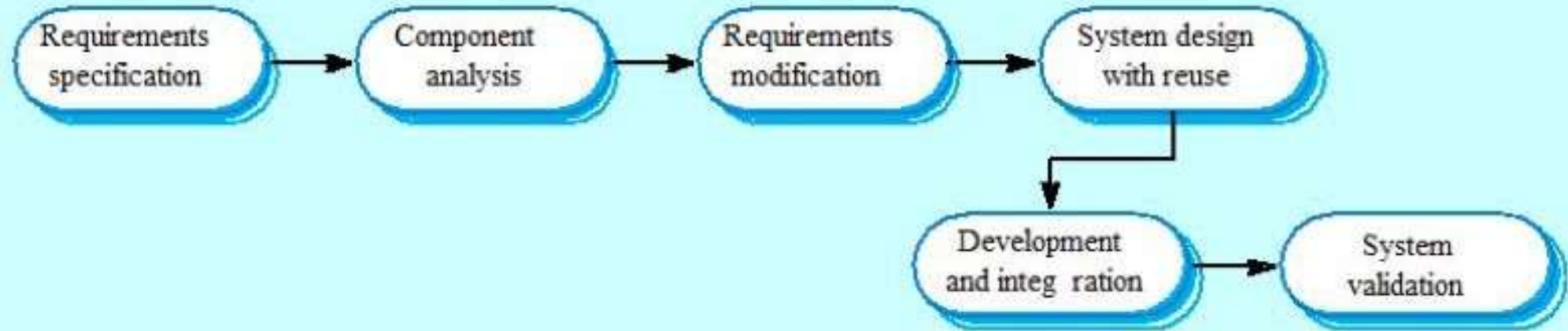
- Based on systematic reuse where systems are integrated from existing components or COTS (Commercial-of-the-shelf) systems.

### **Process stages:**

1. Component analysis
2. Requirements modification
3. System design with reuse
4. Development and integration

- This approach is becoming increasingly used as component standards have emerged

# Reuse - oriented development



# Agenda

- Introduction to Software Engineering
- Software Processes
- ✓ Rapid software development
- Practices you should Know
- Case Study

# **Rapid software development**



# Rapid software development

- Because of rapidly changing business environments, businesses have to respond to new opportunities and competition.
- This requires software and rapid development ,delivery is not often the most critical requirement for software systems.
- Businesses may be willing to accept lower quality software if rapid delivery of essential functionality is possible.

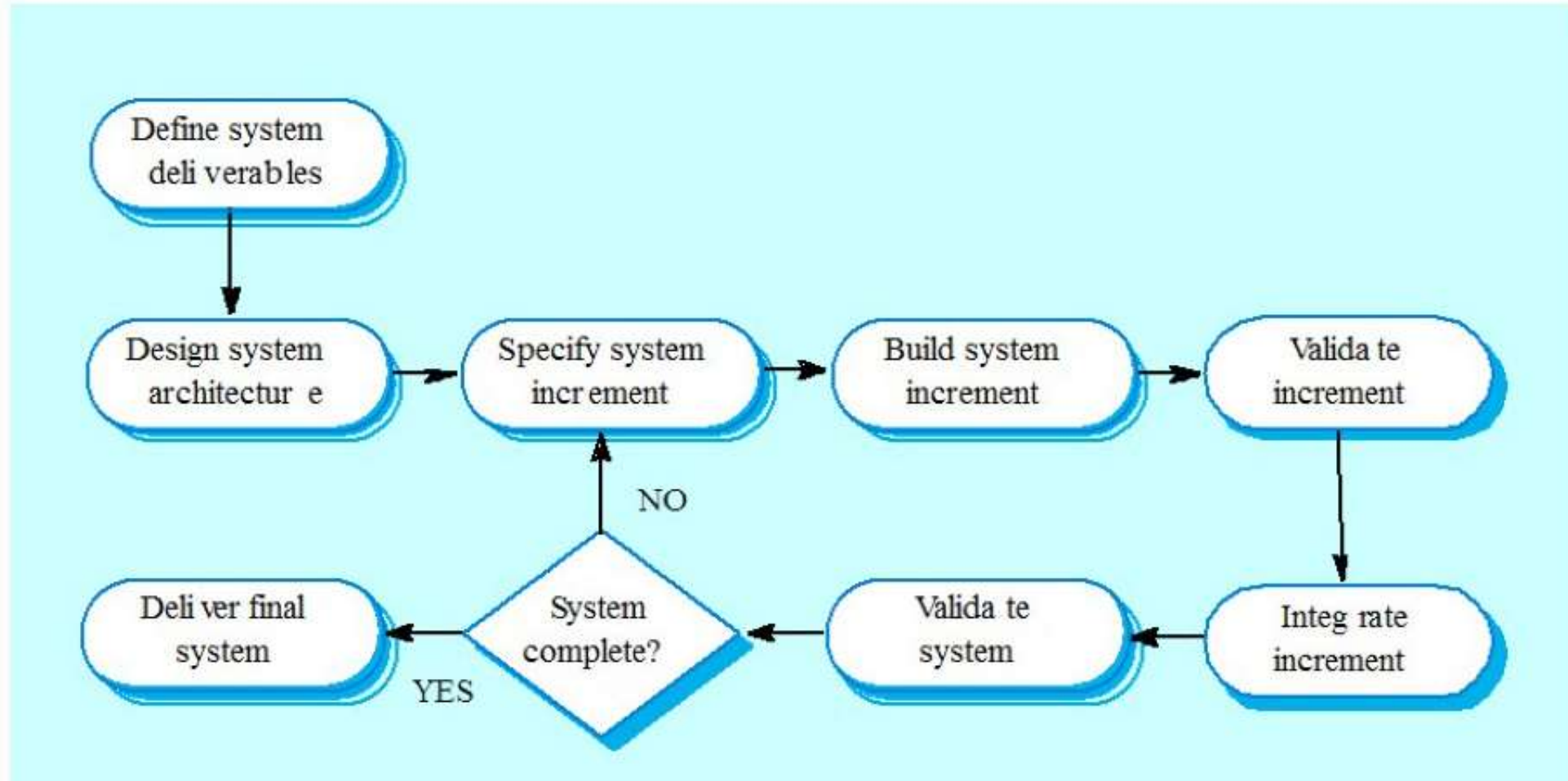
# Requirements

- **Because of** the changing environment, it is often impossible to arrive at a stable, consistent set of system requirements.
- **Therefore** a waterfall model of development is impractical and an approach to development based on iterative specification and delivery is the only way to deliver software quickly.

# Characteristics of RAD processes

- The processes of specification, design and implementation are concurrent. There is no detailed specification and design documentation is minimized.
- The system is developed in a series of increments. End users evaluate each increment and make proposals for later increments.
- System user interfaces are usually developed using an interactive development system.

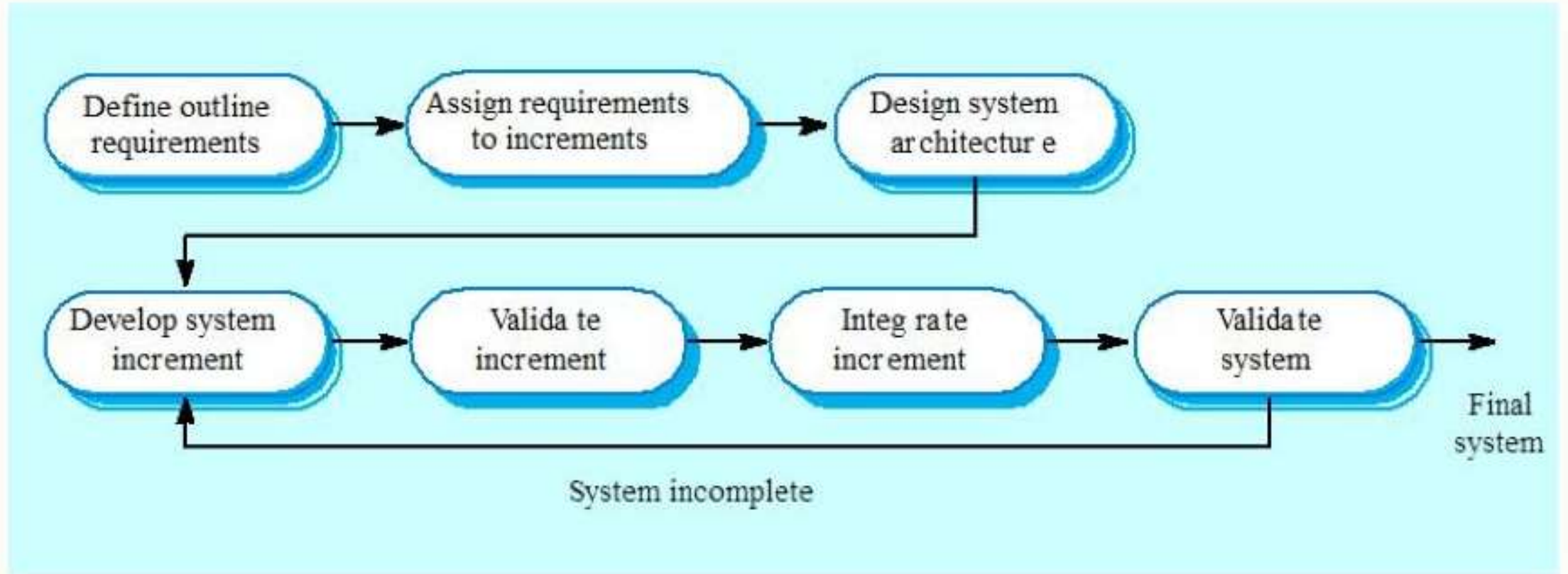
# Iterative development process



# Incremental delivery

- Rather than deliver the system as a single delivery, the development and delivery is broken down into increments with each increment delivering part of the required functionality.
- User requirements are prioritized and the highest priority requirements are included in early increments.
- Once the development of an increment is started, the requirements are frozen though requirements for later increments can continue to evolve.

# Incremental development



# Incremental development advantages

- Customer value can be delivered with each increment so system functionality is available earlier.
- Early increments act as a prototype to help elicit requirements for later increments.
- Lower risk of overall project failure.
- The highest priority system services tend to receive the most testing.

# Advantages of incremental development

- **Accelerated** delivery of customer services. Each increment delivers the highest priority functionality to the customer.
- **User engagement** with the system. Users have to be involved in the development which means the system is more likely to meet their requirements and the users are more committed to the system.



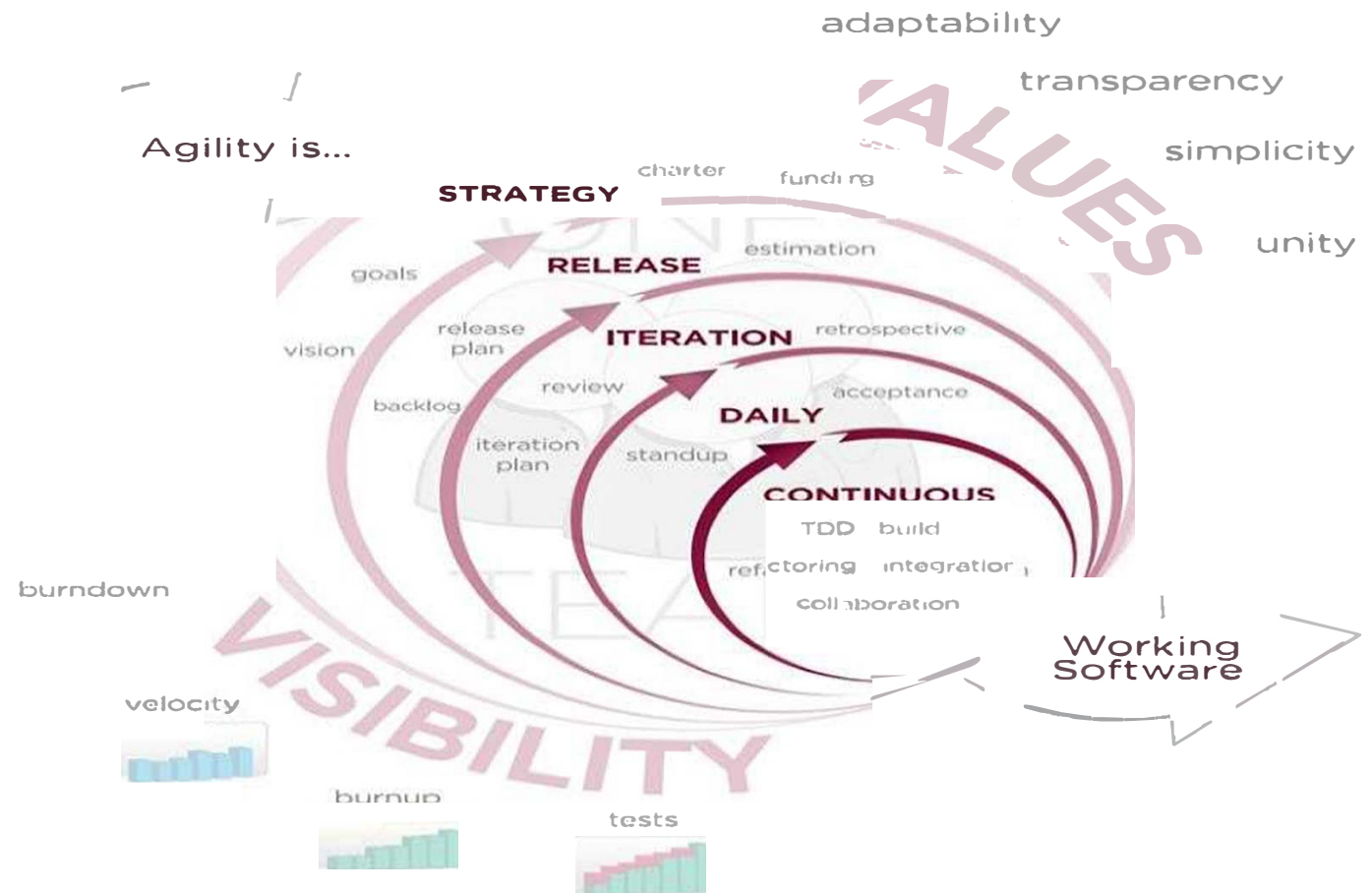
# Problems with incremental development

- Management problems
  - Progress can be hard to judge and problems hard to find because there is no documentation to demonstrate what has been done.
- Contractual problems
  - The normal contract may include a specification; without a specification, different forms of contract have to be used.
- Validation problems
  - Without a specification, what is the system being tested against?
- Maintenance problems
  - Continual change tends to corrupt software structure making it more expensive to change and evolve to meet new requirements.

# Agile methods

- Dissatisfaction with the overheads involved in design methods led to the creation of agile methods. These methods:
  - Focus on the code rather than the design
  - Are based on an iterative approach to software development.
  - Are intended to deliver working software quickly and evolve this quickly to meet changing requirements.
- Agile methods are probably best suited to small/medium-sized business systems or PC products.

# AGILE DEVELOPMENT



## ACCELERATE DELIVERY

# Principles of agile methods

| Principle            | Description                                                                                                                                                                            |
|----------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Customer involvement | The customer should be closely involved throughout the development process. Their role is provide and prioritise new system requirements and to evaluate the iterations of the system. |
| Incremental delivery | The software is developed in increments with the customer specifying the requirements to be included in each increment.                                                                |
| People not process   | The skills of the development team should be recognised and exploited. The team should be left to develop their own ways of working without prescriptive processes.                    |
| Embrace change       | Expect the system requirements to change and design the system so that it can accommodate these changes.                                                                               |
| Maintain simplicity  | Focus on simplicity in both the software being developed and in the development process used. Wherever possible, actively work to eliminate complexity from the system.                |

# Problems with agile methods

- It can be difficult to keep the interest of customers who are involved in the process.
- Team members may be unsuited to the intense involvement that characterizes agile methods.
- Prioritizing changes can be difficult where there are multiple stakeholders.
- Maintaining simplicity requires extra work.
- Contracts may be a problem as with other approaches to iterative development.

# Agenda

- Introduction to Software Engineering
- Software Processes
- Rapid software development
- ✓ Practices you should Know
- Case Study

**Practices you should Know**

# Pair programming

- Programmers work in pairs, sitting together to develop code.
- This helps develop common ownership of code and spreads knowledge across the team.
- It serves as an informal review process as each line of code is looked at by more than 1 person.
- It encourages refactoring as the whole team can benefit from this. Measurements suggest that development productivity with pair programming is similar to that of two people working independently.



# Software prototyping

- A prototype is an initial version of a system used to demonstrate concepts and try out design options.
- **A prototype can be used in:**
  - The requirements engineering process to help with requirements elicitation and validation
  - In design processes to explore options and develop a UI design.
  - In the testing process to run back-to-back tests.

# Benefits of prototyping

- Improved system usability.
- A closer match to users' real needs.
- Improved design quality.
- Improved maintainability.
- Reduced development effort.

# Process activities

- Software specification
- Software design and implementation
- Software validation
- Software Acceptance

# Agenda

- Introduction to Software Engineering
- Rapid software development
- Software Processes
- Practices you should Know
- ✓ Case Study

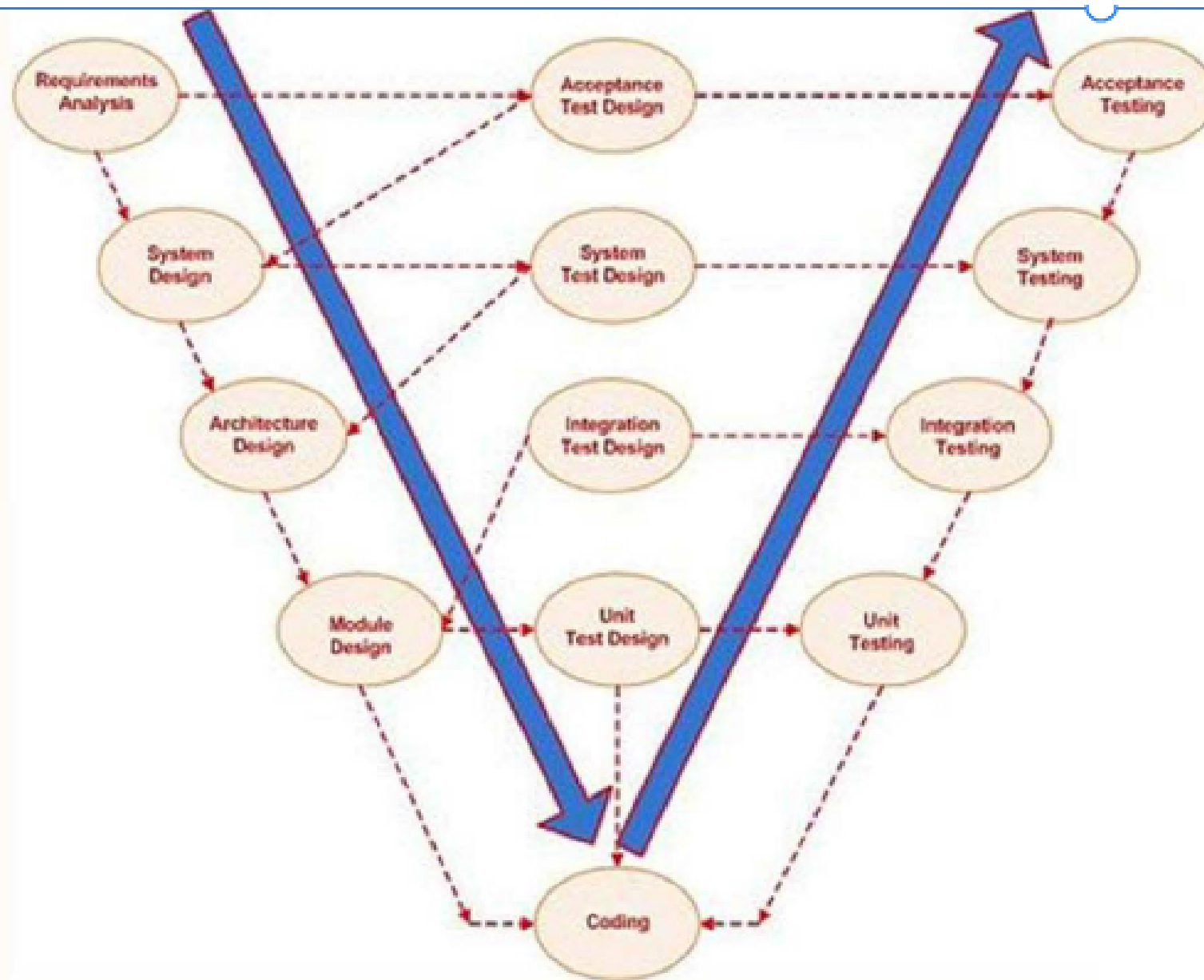
# V-Model

# V-Model

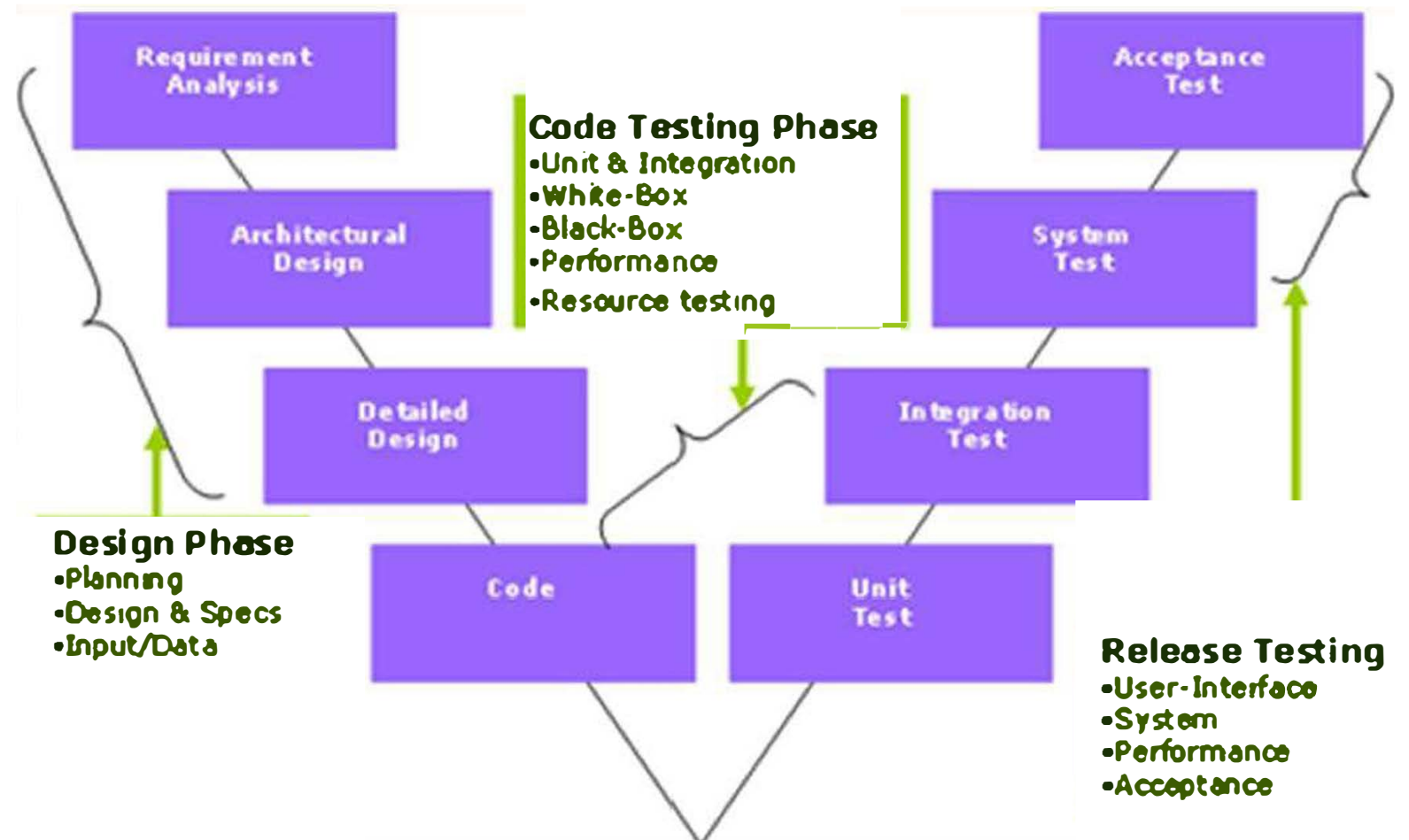
- The V-model represents a software development process (also applicable to hardware development) which may be considered an extension of the waterfall model. Instead of moving down in a linear way, the process steps are bent upwards after the coding phase, to form the typical V shape.

# V-Model

- The V-Model demonstrates the relationships between each phase of the development life cycle and its associated phase of testing. The horizontal and vertical axes represents time or project completeness (left-to-right) and level of abstraction (coarsest-grain abstraction uppermost), respectively.







# V-Model phases

| Phase                                                                                                                                | Testing Activities                                                                                                                                                | Types of Tools                                                                                                                                                 |
|--------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <ul style="list-style-type: none"><li>• Requirement Specification</li><li>• Architectural Design</li><li>• Detailed Design</li></ul> | <ul style="list-style-type: none"><li>• Planning</li><li>• Design &amp; Specs</li><li>• Input / Data</li></ul>                                                    | <ul style="list-style-type: none"><li>• Planning</li><li>• Test Requirements</li><li>• Test Management</li><li>• Test Design</li></ul>                         |
| <ul style="list-style-type: none"><li>• Coding</li><li>• Unit Testing</li><li>• Integration Testing</li></ul>                        | <ul style="list-style-type: none"><li>• Unit &amp; Integration</li><li>• White-Box</li><li>• Black-Box</li><li>• Performance</li><li>• Resource testing</li></ul> | <ul style="list-style-type: none"><li>• Static Analysis</li><li>• Debugging</li><li>• Code Coverage</li><li>• Dynamic Analysis</li><li>• Memory Leak</li></ul> |
| <ul style="list-style-type: none"><li>• System Testing</li><li>• User Acceptance</li></ul>                                           | <ul style="list-style-type: none"><li>• User-Interface</li><li>• System</li><li>• Performance</li><li>• Acceptance</li></ul>                                      | <ul style="list-style-type: none"><li>• Stress/Load</li></ul>                                                                                                  |

## V-Model phases cont.."

- The deliverables from the first 3 phases of the lifecycle forms the input for testing process.
- These deliverables are in the form of specification documents.

# V-Software Life Phases

## V-Model phases

- Requirements Phase -> complete specification what the software is expected to do.
- Requirements analysis

In the Requirements analysis phase, the requirements of the proposed system are collected by analyzing the needs of the user(s).

This phase is concerned with establishing what the ideal system has to perform. However it does not determine how the software will be designed or built.

Usually, the users are interviewed and a document called the user requirements document is generated.

The user requirements document will typically describe the system's functional, interface, performance, data, security, etc. requirements as expected by the user.

It is used by business analysts to communicate their understanding of the system to the users.

The users carefully review this document as this document would serve as the guideline for the system designers in the system design phase.

The user acceptance tests are designed in this phase.

There are different methods for gathering requirements of both soft and hard methodologies including; interviews, questionnaires, document analysis, observation, throw-away prototypes, use cases and status and dynamic views with users.

# V-Software Life Cont.."

- Architectural Design -> identifying entities within the software and their relationships between each other.

## **System Design:**

Systems design is the phase where system engineers analyze and understand the business of the proposed system by studying the user requirements document. They figure out possibilities and techniques by which the user requirements can be implemented. If any of the requirements are not feasible, the user is informed of the issue. A resolution is found and the user requirement document is edited accordingly.

The software specification document which serves as a blueprint for the development phase is generated. This document contains the general system organization, menu structures, data structures etc. It may also hold example business scenarios, sample windows, reports for the better understanding. Other technical documentation like entity diagrams, data dictionary will also be produced in this phase. The documents for system testing are prepared in this phase.

## **Architecture Design:**

The phase of the design of computer architecture and software architecture can also be referred to as high-level design. The baseline in selecting the architecture is that it should realize all which typically consists of the list of modules, brief functionality of each module, their interface relationships, dependencies, database tables, architecture diagrams, technology details etc. The integration testing design is carried out in the particular phase.

# V-Model phases cont.."

- Detailed Design -> where each of the entities scope and behavior are described in detail.

## Detailed "Module" Design:

The module design phase can also be referred to as low-level design. The designed system is broken up into smaller units or modules and each of them is explained so that the programmer can start coding directly. The low level design document or program specifications will contain a detailed functional logic of the module, in pseudo code: database tables, with all elements, including their type and size all interface details with complete API references

all dependency issues

error message listings

complete input and outputs for a module.

The unit test design is developed in this stage.

# V-Model phases cont.."

- Unit Testing -> In computer programming, unit testing is a method by which individual units of source code are tested to determine if they are fit for use.

## **Unit Testing:**

A unit is the smallest testable part of an application. In procedural programming a unit may be an individual function or procedure. Unit tests are created by programmers or occasionally by white box testers. The purpose is to verify the internal logic code by testing every possible branch within the function, also known as test coverage. Static analysis tools are used to facilitate in this process, where variations of input data are passed to the function to test every possible case of execution.

# V-Model phases cont.."

- Integration Testing -> In integration testing the separate modules will be tested together to expose faults in the interfaces and in the interaction between integrated components.
- Testing is usually black box as the code is not directly checked for errors.

## **Validation Testing:**

Validation testing will compare the software specifications “SRS” against the actual system.

After the integration test is completed, the next test level is the Validation test.

Validation testing checks if the integrated product meets the specified requirements. Why is this still necessary after the component and integration tests? The reasons for this are as follows:

In the lower test levels, the testing was done against technical specifications, i.e., from the technical perspective of the software producer. The validation test, though, looks at the system from the perspective of the customer and the future user.

The testers validate whether the requirements are completely and appropriately met.

**Example:** The customer (who has ordered and paid for the system) and the user (who uses the system) can be different groups of people or organizations with their own specific interests and requirements of the system.

Many functions and system characteristics result from the interaction of all system components, consequently, they are only visible on the level of the entire system and can only be observed and tested there.



# V-Model phases cont.."

- User Acceptance Testing-> Is the phase of testing used to determine whether a system satisfies the requirements specified in the requirements analysis phase.
- The acceptance test phase is the phase used by the customer to determine whether to accept the system or not.

## **User Acceptance Testing:**

Acceptance testing is the phase of testing used to determine whether a system satisfies the requirements specified in the requirements analysis phase. The acceptance test design is derived from the requirements document.

The acceptance test phase is the phase used by the customer to determine whether to accept the system or not.

## **Acceptance testing helps:**

to determine whether a system satisfies its acceptance criteria or not.

to enable the customer to determine whether to accept the system or not.

to test the software in the "real world" by the intended audience.

## **Purpose of acceptance testing:**

to verify the system or changes according to the original needs.

# V-Model phases cont.."

## **Acceptance Testing Procedures:-**

Define the acceptance criteria:

Functionality requirements.

Performance requirements.

Interface quality requirements.

Overall software quality requirements.

Develop an acceptance plan:

Project description.

User responsibilities.

Acceptance description.

Execute the acceptance test plan.

## **Release Testing:**

Release testing is a phase that determines if the software is suitable for the organization of the end -user.

How is compatibility with other systems ensured?

Is the performance of the software optimized?

## **V-Model phases**

# Software Case Tools

- CASE stands for **C**omputer **A**ided **S**oftware **E**ngineering. It means, development and maintenance of software projects with help of various automated software tools.
- CASE Tools such as :
  - ✓ Analysis tools
  - ✓ Design tools
  - ✓ Project management tools
  - ✓ Database Management tools
  - ✓ Documentation tools .

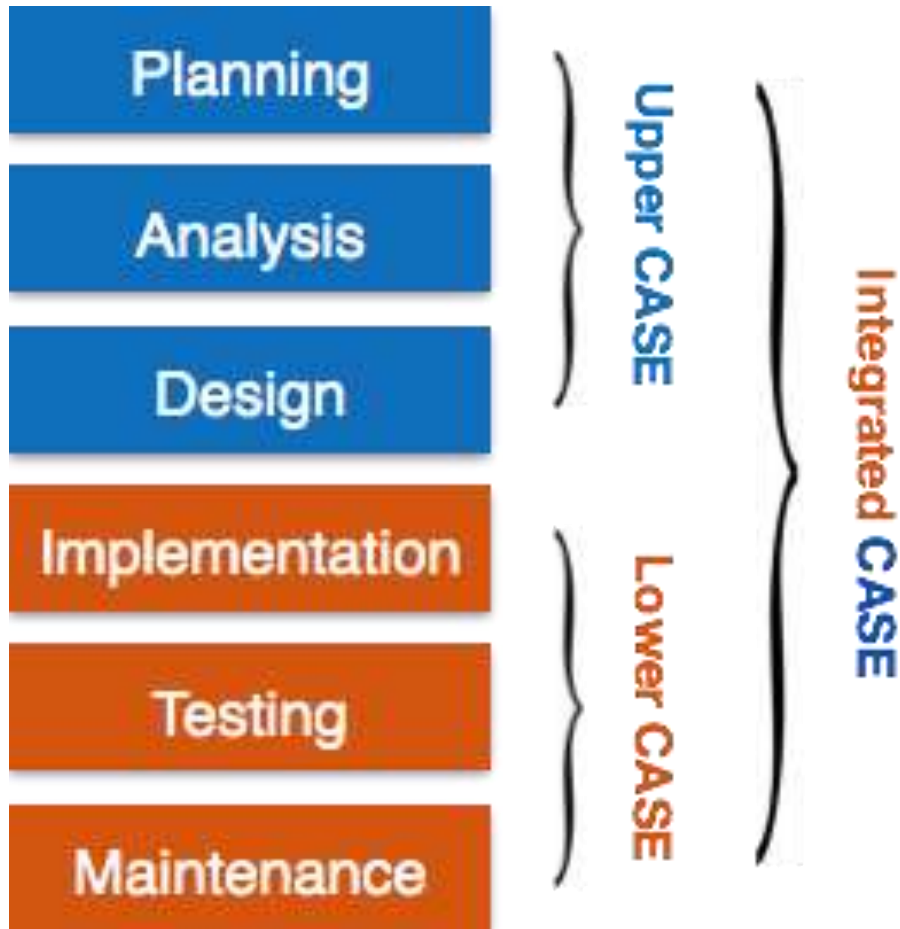
# Components of CASE Tools

- CASE tools can be broadly divided into the following parts based on their use at a particular SDLC stage:

- **Central Repository :**

CASE tools require a central repository, which can serve as a source of common, integrated and consistent information. Central repository **is a central place of storage** where product specifications, requirement documents, related reports and diagrams, other useful information regarding management is stored. Central repository also serves as data dictionary.

# Components of CASE Tools



- **Upper Case Tools** - Upper CASE tools are used in planning, analysis and design stages of SDLC.
- **Lower Case Tools** - Lower CASE tools are used in implementation, testing and maintenance.
- **Integrated Case Tools** - Integrated CASE tools are helpful in all the stages of SDLC, from Requirement gathering to Testing and documentation.

# Case Tools Types

## ➤ Diagram Tools:

These tools are used to represent system components, data and control flow among various software components and system structure in a graphical form. For example, Flow Chart Maker tool for creating state-of-the-art flowcharts.

- **UML** is a standard language for specifying, visualizing, constructing, and documenting the artifacts of software systems.
- **Unified Modelling Language (UML)**

# Case Tools Types

## ➤ Process Modeling Tools

Process modeling is method to create software process model, which is used to develop the software. Process modeling tools help the managers to choose a process model or modify it as per the requirement of software product.

For example:

**EPF Composer**

# Case Tools Types

## ➤ Project Management Tools

- These tools are used for project planning, cost and effort estimation, project scheduling and resource planning. Managers have to strictly comply project execution with every mentioned step in software project management. Project management tools help in storing and sharing project information in real-time throughout the organization. For example:

**Creative Pro Office, Trac Project, Basecamp.**



# Case Tools Types

## ➤ Documentation Tools

Documentation tools generate documents for technical users and end users. Technical users are mostly in-house professionals of the development team who refer to system manual, reference manual, training manual, installation manuals etc. The end user documents describe the functioning and how-to of the system such as user manual.

For example:

**Doxygen, DrExplain, Adobe RoboHelp** for documentation.

# Case Tools Types

## ➤ Analysis Tools :

These tools help to gather requirements, automatically check for any inconsistency, inaccuracy in the diagrams, data redundancies or erroneous omissions.

For example :

**Accept 360, Accompa, CaseComplete** for requirement analysis.

# Case Tools Types

## ➤ Design Tools

These tools help software designers to design the block structure of the software, which may further be broken down in smaller modules using refinement techniques. These tools provides detailing of each module and interconnections among modules.

For example :

Animated Software Design

# Case Tools Types

## ➤ Configuration Management Tools

An instance of software is released under one version. Configuration Management tools deal with

- ✓ Version and revision management
- ✓ Baseline configuration management
- ✓ Change control management

CASE tools help in this by automatic tracking, version management and release management.

For example:

**Fossil, Git, Accu REV.**

# Case Tools Types

## ➤ Programming Tools

- These tools consist of programming environments like **IDE** (Integrated Development Environment), in-built modules library and simulation tools. These tools provide comprehensive aid in building software product and include features for simulation and testing.

For example :

**Cscope** to search code in **C**, **Eclipse**.

# Case Tools Types

## ➤ Prototyping Tools

- Software prototype is simulated version of the intended software product. Prototype provides initial look and feel of the product and simulates few aspect of actual product.
- Prototyping CASE tools essentially come with graphical libraries. They can create hardware *independent user interfaces and design*. These tools help us to *build rapid prototypes based on existing information*. In addition, they provide *simulation of software prototype*.

For example:

**Serena** prototype composer, **Mockup** Builder.

# Case Tools Types

## ➤ Web Development Tools

- Web tools provide live preview of what is being developed and how will it look after completion.

For example:

**Fontello, Adobe Edge Inspect, Foundation 3, Brackets.**

# Case Tools Types

## ➤ Quality Assurance Tools

QA tools consist of configuration and change control tools and software testing tools.

For example:

**SoapTest, AppsWatch, JMeter.**



# Case Tools Types

## ➤ Maintenance Tools:

- Software maintenance includes modifications in the software product after it is delivered. Automatic logging and error reporting techniques, automatic error ticket generation and root cause Analysis are few CASE tools, which help software organization in maintenance phase of SDLC.

For example:

**Bugzilla** for defect tracking, **HP Quality Center**.

Thank you